

## 자바스크립트 객체, 함수, DOM 정리

### 오늘의 강의 학습 요약

금일 강의는 자바스크립트 객체(리터럴/JSON 형태)의 핵심 문법을 체계적으로 정리한 뒤, 문자열↔객체 변환 (JSON.parse / JSON.stringify)까지 확장했습니다. 이후 스프레드 연산자를 통한 배열 복사, 추가, 연결과 "새 배열 생성(불변성)" 관점을 프론트엔드 렌더링과 연결해 설명했습니다.

후반부에는 함수 선언식/표현식의 생성 시점 차이(호이스팅), 인자/파라미터 불일치, 디폴트 파라미터, 레스트 파라미터, 스프레드 인자 전달을 정리하고, 1급 객체/콜백/화살표 함수/제너레이터/즉시 실행 함수까지 함수 관련 개념을 확장했습니다. 마지막으로 이벤트 처리(인라인, DOM0 고전, DOM2)와 이벤트 객체, 이벤트 전파/방지, DOM 접근(querySelector/All) 및 innerHTML, innerText, value 기반의 실습 흐름으로 마무리했습니다.

## 1. 객체(JSON) 핵심 문법과 접근

객체 리터럴은 중괄호 기반의 키-밸류 구조이며, 도트/연관배열 방식으로 속성과 함수를 접근하고, `this` 로 객체 내부 속성을 참조합니다.

객체(강의에서는 JSON이라고도 표현됨)는 `{ key: value }` 형태의 키-밸류 구조입니다. 밸류에는 문자열, 숫자, 배열, 중첩 객체, 함수까지 “모든 타입”이 들어갈 수 있으므로, 객체는 데이터 저장소이면서 동작(메소드)을 함께 묶는 구조로 사용됩니다.

속성 접근은 기본적으로 도트 표기법을 사용합니다. 예를 들어 `person.name`, `person.age` 처럼 접근하며, 함수(메소드) 밸류는 `person.getName()` 처럼 괄호를 붙여 호출합니다. 이때 getter처럼 “값만 반환”하는 함수는 파라미터가 없을 수 있고, setter처럼 “값을 설정”하는 함수는 파라미터를 받아 내부 상태를 바꿉니다.

객체 내부에서 자신의 속성을 참조할 때는 `this` 가 중요합니다. 자바처럼 생략하는 개념으로 접근하면 안 되며, 객체 내부에서 `name` 을 읽거나 변경하려면 `this.name` 처럼 명시적으로 작성해야 내부 속성으로 연결됩니다.

속성 접근은 도트 외에도 연관 배열(대괄호) 표기법이 가능합니다. 즉 `person["name"]` 처럼 키를 문자열로 넣어 접근할 수 있으며, 이 방식의 핵심 장점은 “키를 변수로 치환”할 수 있다는 점입니다. 예를 들어 `let key = "name"; person[key]` 처럼 동적으로 속성을 선택할 수 있지만, `person.key` 는 실제로 `key` 라는 이름의 속성을 찾게 되므로 의도와 달라집니다.

객체 리터럴에서는 자주 쓰는 축약 문법이 있습니다. 키와 밸류로 들어가는 변수명이 같으면 `name: name` 대신 `name` 만 작성할 수 있고, 함수 밸류도 `getName: function(){}`  대신 메소드 축약 형태로 `getName(){}` 처럼 작성할 수 있습니다.

아울러 getter/setter 문법( `get` , `set` )을 사용하면 함수처럼 호출하지 않고 “속성처럼” 읽고 쓰는 형태로 통일할 수 있습니다. 예를 들어 `get userName()` 과 `set userName(v)` 를 정의하면, 읽을 때는 `person.userName` , 쓸 때는 `person.userName = "홍길동"` 처럼 사용되며, 이때 호출 여부에 따라 getter/setter가 선택됩니다. 다만 기존 속성명과 충돌이 날 수 있으므로 이름을 분리하는 설계가 필요합니다.

키 자체도 변수로 동적으로 만들 수 있습니다. 객체 리터럴에서 키를 변수로 쓰려면 반드시 대괄호 표기(Computed Property Name)를 사용해야 하며, 문자열 결합을 통해 `name01` 처럼 가공된 키 생성도 가능합니다.

| 구분         | 도트 표기법                | 연관 배열 표기법(대괄호)                         |
|------------|-----------------------|----------------------------------------|
| 기본 형태      | <code>obj.name</code> | <code>obj["name"]</code>               |
| 키를 변수로 사용  | 불가능합니다.               | 가능합니다.                                 |
| 키 가공(동적 키) | 불가능합니다.               | <code>obj[key + "01"]</code> 처럼 가능합니다. |

```
const key = "name";
const person = {
  name: "홍길동",
  getName() { return this.name; },
};

console.log(person[key]); // "홍길동"
console.log(person.getName()); // "홍길동"
```

핵심 키워드

#객체리터럴

#this

#도트접근

#연관배열접근

#get/set

## 2. JSON 문자열 변환(parse/stringify)

문자열로 표현된 JSON/배열을 실제 객체/배열로 변환할 때는 `JSON.parse`, 반대로 문자열로 직렬화할 때는 `JSON.stringify` 를 사용합니다.

실무에서는 “진짜 객체/배열”이 아니라, 문자열 형태로 전달된 JSON이나 배열을 자주 다루게 됩니다. 이 경우 문자열을 그대로 쓰면 속성 접근이나 배열 메소드 사용이 불가능하므로, 먼저 파싱하여 객체/배열로 바꾸어야 합니다.

`JSON.parse()` 는 JSON 문자열 또는 배열 문자열을 실제 자바스크립트 객체/배열로 변환합니다. 반대로 `JSON.stringify()` 는 객체/배열을 JSON 문자열로 변환합니다. 강의에서는 숫자 문자열을 `parseInt` 로 바꾸듯, JSON도 `parse/stringify`가 그만큼 빈번하게 쓰인다고 강조했습니다.

```
const arrStr = '["A","B","C"]';
const arr = JSON.parse(arrStr);           // ["A","B","C"]

const obj = { name: "홍길동", age: 20 };
const jsonStr = JSON.stringify(obj);      // '{"name":"홍길동","age":20}'
```

핵심 키워드

#JSON.parse

#JSON.stringify

#직렬화

#역직렬화

#문자열 JSON

### 3. 스프레드 연산자와 불변성(새 배열 생성)

스프레드( `...` )는 배열/문자열을 요소 단위로 펼쳐 복사, 추가, 연결을 만들며, 새 배열을 생성해 프론트엔드 렌더링에서 변경 감지에 유리합니다.

스프레드 연산자는 `...` (점 3개)로 표현하며, “뿌린다(spread)”는 의미처럼 배열의 요소를 낱개로 펼쳐줍니다. 대표 용도는 배열 복사, 기존 배열에 요소 추가, 두 배열 연결입니다.

스프레드로 복사하면 “새 배열”이 만들어진다는 점이 중요합니다. 단순 대입( `let b = a` )은 주소를 공유하므로 한쪽 수정이 원본에 영향을 줍니다. 반면 스프레드 복사는 다른 주소를 갖는 새 배열이므로 원본 불변성을 유지하기 쉬워집니다.

프론트엔드 프레임워크에서는 상태 변경을 “주소(참조) 변경”으로 감지하는 경우가 많아, 요소를 직접 수정하면 변경을 인지하지 못할 수 있습니다. 스프레드는 쉽게 새 주소(새 배열)를 만들 수 있어 리렌더링 트리거 관점에서 자주 사용됩니다.

| 사용 목적  | 예시                                   | 결과 개념                                  |
|--------|--------------------------------------|----------------------------------------|
| 복사     | <code>const b = [...a];</code>       | 새 배열 생성(주소 분리)입니다.                     |
| 추가     | <code>const c = [...a, 1, 2];</code> | 기존 요소 + 새 요소 조합입니다.                    |
| 연결     | <code>const d = [...a, ...b];</code> | 두 배열 요소를 한 배열로 합칩니다.                   |
| 문자열 펼침 | <code>[..."ABC"]</code>              | <code>["A", "B", "C"]</code> 처럼 펼쳐집니다. |

핵심 키워드    `#스프레드`    `#불변성`    `#얕은복사`    `#배열연결`    `#리렌더링`



## 4. 함수 선언식, 표현식과 파라미터 확장

함수는 선언식/표현식으로 정의되며 생성 시점이 다르고, 인자 개수 불일치가 허용되며, 디폴트, 레스트, 스프레드로 이를 제어합니다.

함수는 기능 처리 단위이며 "호출해야 실행"된다는 점은 메소드와 동일합니다. 다만 문법적으로 함수 선언식(이름 있는 함수)과 함수 표현식(익명 함수를 변수에 저장)으로 구분됩니다.

함수 선언식은 스크립트가 시작될 때 생성되는 특성이 있어, 정의보다 먼저 호출이 가능할 수 있습니다(호이스팅 관점). 반면 함수 표현식은 "해당 실행 문장에 도달했을 때" 함수가 생성되므로, 정의 이전 호출은 에러가 발생합니다.

자바스크립트는 자바와 달리 파라미터 개수와 인자 개수가 일치하지 않아도 호출이 가능합니다. 인자가 부족하면 남은 파라미터는 `undefined` 가 되며, 인자가 많으면 초과 인자는 기본적으로 버려집니다.

이 동작을 보완하기 위해 디폴트 파라미터(기본값 지정)로 `undefined` 를 방지할 수 있고, 레스트 파라미터 (`...rest`)로 초과 인자를 배열로 모아 데이터 손실을 방지할 수 있습니다. 또한 호출 시 배열에 들어 있는 값을 날개 인자로 전달하려면 스프레드(`func(...arr)`)를 사용합니다.

| 기능       | 문법 예시                                 | 핵심 효과                |
|----------|---------------------------------------|----------------------|
| 디폴트 파라미터 | <code>function f(a=10){}</code>       | 인자 누락 시 기본값이 적용됩니다.  |
| 레스트 파라미터 | <code>function f(a, ...rest){}</code> | 초과 인자가 배열로 수집됩니다.    |
| 스프레드 호출  | <code>f(...arr)</code>                | 배열 요소가 인자로 펼쳐 전달됩니다. |

```
function sum(a = 0, b = 0, ...rest) {  
  return a + b + rest.reduce((acc, v) => acc + v, 0);  
}  
console.log(sum(10));           // 10  
console.log(sum(10, 20, 30));  // 60
```

핵심 키워드

#함수선언식

#함수표현식

#디폴트파라미터

#레스트파라미터

#호이스팅

## 5. 1급 객체와 콜백, 화살표 함수

함수는 1급 객체로 변수 저장, 인자 전달, 리턴이 가능하며, 콜백은 “함수 이름만 전달”해 시스템/다른 함수가 적절한 시점에 실행하게 하는 패턴입니다.

1급 객체(First-class)란 함수를 데이터처럼 다룰 수 있어 변수에 저장하고, 함수 호출 시 인자로 전달하고, 함수의 반환값으로도 리턴할 수 있는 성질을 의미합니다. 이 관점이 있어야 `setTimeout(fn, delay)` 처럼 함수가 인자로 들어가는 패턴이 자연스럽게 이해됩니다.

콜백은 “내가 직접 실행하지 않고”, 실행되어야 할 함수의 참조(이름)를 전달해 다른 코드가 적절한 시점에 호출하도록 만드는 방식입니다. 이때 `()` 를 붙이면 즉시 실행이 되어 이벤트/타이머의 의도와 달라지므로, “괄호를 치지 않는다”는 점이 핵심입니다.

콜백을 사용하면 동일한 함수(예: `xx`)에 서로 다른 동작(예: `yy`, `zz`)을 주입해 재사용성을 크게 높일 수 있습니다. 이는 이벤트 처리에서도 동일하게 반복되는 핵심 패턴입니다.

화살표 함수(arrow function)는 함수 표현식의 축약 표현입니다. `function` 키워드를 생략하고 `=>` 를 사용하며, 파라미터 1개면 괄호를 생략할 수 있고, 실행문 1줄이면 중괄호를 생략할 수 있습니다. 또한 단일 리턴 표현식이면 `return` 도 생략 가능합니다.

| 구분      | 핵심 문법                               | 핵심 포인트                         |
|---------|-------------------------------------|--------------------------------|
| 콜백 전달   | <code>btn.onclick = handler;</code> | 괄호 없이 참조만 전달합니다.               |
| 화살표 기본  | <code>const f = () =&gt; {}</code>  | 함수 표현식 축약입니다.                  |
| 단일 파라미터 | <code>x =&gt; x + 1</code>          | 괄호 생략이 가능합니다.                  |
| 단일 리턴   | <code>() =&gt; 10</code>            | <code>return</code> 생략이 가능합니다. |

핵심 키워드

#1급객체

#콜백

#setTimeout

#화살표함수

#함수참조

## 6. 제너레이터와 즉시 실행 함수

제너레이터는 `function*` 과 `yield` 로 실행을 분할하며 `next()` 로 단계 실행하고, 즉시 실행 함수 (IIFE)는 정의 직후 자동 실행됩니다.

제너레이터 함수는 함수명 앞에 `*` 를 붙여 정의합니다. 일반 함수와 달리 호출 즉시 본문이 실행되지 않고, 제너레이터 객체를 반환합니다. 실제 실행은 반환된 객체의 `next()` 를 호출할 때 진행됩니다.

제너레이터는 `yield` 를 기준으로 실행을 멈추고 재개할 수 있어, 작업을 “나눠서” 처리해야 하는 경우에 유용합니다. 강의에서는 대량 데이터(수억 라인)를 한 번에 처리하지 않고 일정 단위로 끊어 처리하는 시나리오를 예로 들어, `next()` 호출마다 일정 구간씩 처리하는 흐름을 설명했습니다.

즉시 실행 함수(IIFE)는 함수 표현식을 괄호로 감싸 “정의하자마자 호출”하는 형태입니다. 자바의 `static` 블록처럼 초기화 시점에 자동 실행되는 패턴으로 이해할 수 있습니다.

```
function* gen() {  
  yield 10;  
  yield 20;  
  yield 30;  
}  
const it = gen();  
console.log(it.next()); // 10  
console.log(it.next()); // 20
```

핵심 키워드

#제너레이터

#yield

#next

#IIFE

#function\*



## 7. 이벤트 처리: 인라인, DOM0, DOM2와 이벤트 객체

이벤트는 동작이며 이벤트 소스(발생 위젯), 이벤트 타입(종류), 이벤트 핸들러(처리 함수)로 구성되고, 연결 방식은 인라인, 고전(DOM0), DOM2(addEventListener)로 구분됩니다.

이벤트는 사용자 또는 시스템이 발생시키는 “동작”입니다. 버튼 클릭, 마우스 오버/아웃, 키다운/키업, 포커스/블러, 체인지 등이 대표적입니다. 이벤트가 발생한 요소를 이벤트 소스라고 하며, 이벤트 발생 시 실행되는 함수를 이벤트 핸들러라고 합니다.

인라인 방식은 HTML 태그 속성으로 이벤트를 연결합니다. 속성명은 항상 `on` 으로 시작하고 뒤에 이벤트 타입이 붙습니다(예: `onclick`, `onmouseover`). 로드처럼 시스템이 발생시키는 이벤트도 있으며, `onload` 는 HTML이 렌더링된 뒤 자동 실행되는 대표 사례입니다.

고전 방식(DOM Level 0, 고정 방식)은 태그 밖에서 자바스크립트로 요소를 찾아 (`document.querySelector`) 속성으로 이벤트를 연결합니다. 이때 가장 흔한 실수는 (1) DOM이 생성되기 전 접근하여 `null` 이 되는 문제와 (2) 핸들러에 `()` 를 붙여 즉시 실행해버리는 문제입니다. 해결은 스크립트를 요소 아래에 두거나, `window.onload` 에서 DOM 생성 이후 연결하도록 해야 하며, 이벤트 핸들러는 “콜백으로 참조만 전달”해야 합니다.

DOM Level 2는 `addEventListener` 메소드를 사용해 이벤트를 연결합니다. 첫 번째 인자는 이벤트 타입 문자열(예: `"click"`)이며, `on` 을 붙이지 않습니다. 두 번째 인자는 콜백 함수 참조입니다.

이벤트가 발생하면 브라우저가 이벤트 객체를 자동 생성하며, 핸들러에서 이를 통해 발생 소스(target) 등 다양한 정보를 확인할 수 있습니다. 버튼의 라벨(innerText), 체크박스의 checked 상태 등도 이벤트 객체의 `target` 을 통해 접근 가능합니다.

| 방식      | 연결 형태                                          | 주의점                                       |
|---------|------------------------------------------------|-------------------------------------------|
| 인라인     | <code>&lt;button onclick="fn()"&gt;</code>     | HTML과 JS가 결합되어 관리성이 떨어질 수 있습니다.           |
| DOM0 고전 | <code>el.onclick = fn;</code>                  | DOM 생성 이후 연결해야 하며 괄호를 붙이면 안 됩니다.          |
| DOM2    | <code>el.addEventListener("click", fn);</code> | 이벤트 타입은 문자열이며 <code>on</code> 을 붙이지 않습니다. |

```
window.onload = () => {  
  const btn = document.querySelector("#x");  
  btn.onclick = ok; // ok()가 아니라 ok를 전달합니다.  
};  
  
function ok(e) {  
  console.log(e.target.innerText);  
}
```

## 핵심 키워드

#이벤트소스

#이벤트핸들러

#event.target

#addEventListener

#window.onload

## 8. 이벤트 전파와 stopPropagation

중첩 요소에서 자식 이벤트가 부모로 전파될 수 있으며, `event.stopPropagation()` 으로 전파를 차단할 수 있습니다.

중첩된 레이아웃(부모 DIV 안에 자식 DIV)이 있는 경우, 자식 요소를 클릭하면 자식의 이벤트뿐 아니라 부모의 이벤트까지 함께 동작하는 상황이 발생할 수 있습니다. 이를 이벤트 전파라고 합니다.

원하는 UX가 “자식을 클릭했을 때 자식만 반응”이라면, 자식 핸들러에서 `event.stopPropagation()` 을 호출하여 부모로 전달되는 전파를 차단해야 합니다. 강의에서는 부모(A)와 자식(B)에 클릭 이벤트를 각각 걸었을 때, B를 눌러도 A가 함께 반응하는 문제를 실습으로 확인하고, 차단 후 B만 출력되는 흐름을 확인했습니다.

핵심 키워드

#이벤트전파

#버블링

#stopPropagation

#중첩DIV

#이벤트객체

## 9. DOM 처리: querySelector와 콘텐츠/value 접근

DOM 처리는 JS에서 HTML 요소 객체를 조회, 수정하는 작업이며, `querySelector/All` 로 접근한 뒤 `innerHTML/innerText/value` 등 속성으로 값을 읽고 설정합니다.

DOM 처리는 자바스크립트에서 HTML 태그를 "객체로 접근"해 조회/수정/추가하는 작업입니다. 강의에서는 오래된 `getElementById` 도 언급했지만, 핵심은 CSS 선택자를 그대로 쓰는 `document.querySelector` 와 `document.querySelectorAll` 입니다.

`querySelector` 는 조건에 맞는 첫 번째 요소 1개를 반환하고, `querySelectorAll` 은 여러 개를 찾아 컬렉션 (배열 형태로 활용 가능)으로 반환합니다. 반환되는 것은 태그 문자열이 아니라, 해당 태그에 대응하는 요소 객체이며, 객체의 속성/메소드를 활용해 화면을 변경합니다.

콘텐츠를 다루는 핵심 속성은 `innerHTML` 과 `innerText` 입니다. `innerHTML` 은 태그를 포함한 HTML 조각을 읽고/설정하며, `innerText` 는 텍스트만 읽고/설정합니다. 반면 사용자 입력 기반 태그(대표적으로 `input`, `select`, `textarea`)는 콘텐츠 영역이 아니라 입력값을 가지므로 `value` 로 읽고 설정합니다.

강의 실습에서는 (1) P 태그의 콘텐츠를 읽어 다른 영역에 출력하고, (2) P 태그 콘텐츠를 수정하며, (3) input에 입력한 값을 읽어 출력하거나 길이를 세고, (4) 두 입력값을 숫자로 변환해 산술 연산하고, (5) 수량(value)과 가격(value)을 곱해 총액을 출력(innerHTML)하는 흐름을 다뤘습니다.

| 대상            | 값 접근/설정                       | 의미                 |
|---------------|-------------------------------|--------------------|
| 시작/종료 태그의 콘텐츠 | <code>innerText</code>        | 텍스트만 다룹니다.         |
| 시작/종료 태그의 콘텐츠 | <code>innerHTML</code>        | 태그 포함 HTML을 다룹니다.  |
| 사용자 입력 태그     | <code>value</code>            | 입력/선택된 값을 다룹니다.    |
| 다중 선택         | <code>querySelectorAll</code> | 여러 요소를 컬렉션으로 받습니다. |

핵심 키워드

`#querySelector`

`#querySelectorAll`

`#innerHTML`

`#innerText`

`#value`

## 10. 객체분해할당(배열, 객체)과 응용

구조 분해 할당은 배열은 대괄호, 객체는 중괄호로 분해해 변수에 즉시 할당하며, 레스트, 디폴트, 별칭, 일부 추출, 스왑 등에 활용됩니다.

구조 분해 할당(디스트럭처링)은 배열/객체의 값을 쉽게 변수로 꺼내는 문법입니다. 배열은 대괄호로, 객체는 중괄호로 분해합니다. 기존처럼 인덱스 접근( `arr[0]` )이나 도트 접근( `obj.name` )을 반복하지 않고 한 줄로 추출할 수 있습니다.

배열 분해는 순서 기반이며, 분해 변수 개수가 적으면 나머지 요소는 무시됩니다. 반대로 변수 개수가 더 많으면 부족한 값은 `undefined` 가 되며, 디폴트 값을 통해 이를 방지할 수 있습니다. 또한 레스트 연산자를 사용하면 남은 요소를 배열로 모을 수 있고, 특정 위치만 필요하면 쉼표로 건너뛰어 선택할 수 있습니다. 두 변수 값을 교환하는 스왑도 임시 변수 없이 가능합니다.

객체 분해는 키 기반이며, 순서는 중요하지 않습니다. 중괄호 안에는 "키 이름"을 적어야 하며, 키가 없으면 `undefined` 가 됩니다. 별칭이 필요하면 `key: alias` 형태로 변수명을 바꿀 수 있고, 디폴트 값도 함께 지정할 수 있습니다. 이미 선언된 변수에 나중에 분해 할당할 때는 전체를 괄호로 감싸는 문법상의 주의가 있습니다.

| 구분    | 기본 형태                                      | 핵심 포인트              |
|-------|--------------------------------------------|---------------------|
| 배열 분해 | <code>const [a,b] = arr;</code>            | 순서 기반입니다.           |
| 객체 분해 | <code>const {name, age} = obj;</code>      | 키 기반이며 순서 무관입니다.    |
| 별칭    | <code>const {name: userName} = obj;</code> | 키는 유지하고 변수명만 변경합니다. |
| 레스트   | <code>const [a, ...rest] = arr;</code>     | 남는 요소를 배열로 모읍니다.    |

```
const obj = { name: "홍길동", age: 20 };
const { name: userName, age = 0 } = obj;

const arr = [10, 20, 30, 40];
const [a, b, ...rest] = arr; // a=10, b=20, rest=[30,40]
```

### 핵심 키워드

#구조분해할당

#디스트럭처링

#별칭

#레스트

#디폴트값